Solution: Number Frequency

Problem Statement

Write a Recursive Solution to Count occurrences of an Element in an Array.

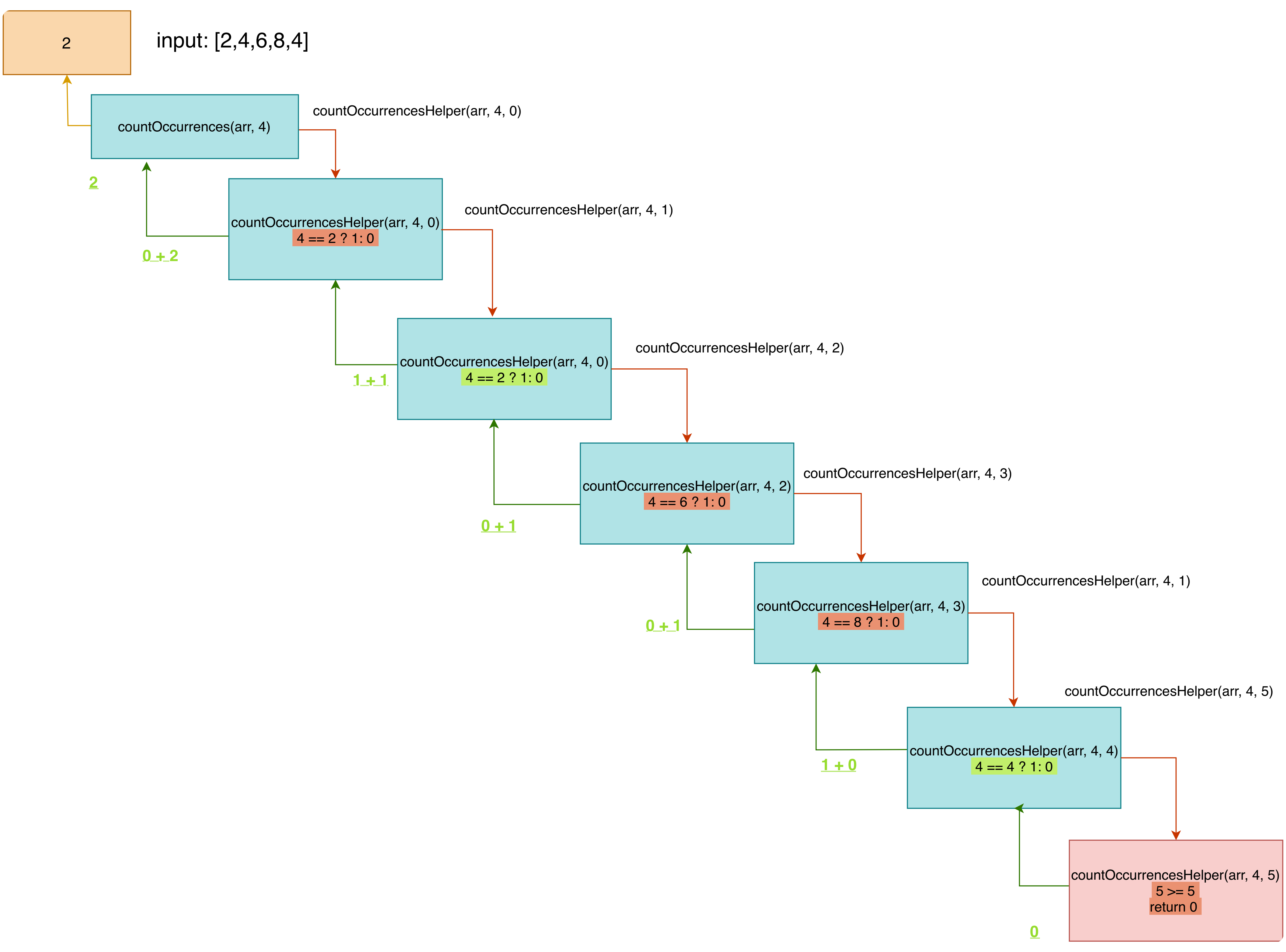
Given an array of integers and a key element, write a recursive solution to count the occurrences of the key element in the array.

Examples:

Sr#	Array	Input Key	Output	Description
1	[2, 4, 6, 8, 4]	4	2	The key element 4 occurs twice in the array.
2	[1, 3, 5, 7, 9]	2	0	The key element 2 does not exist in the array.
3	[1, 2, 2, 2, 3]	2	3	The key element 2 occurs three times in the array.

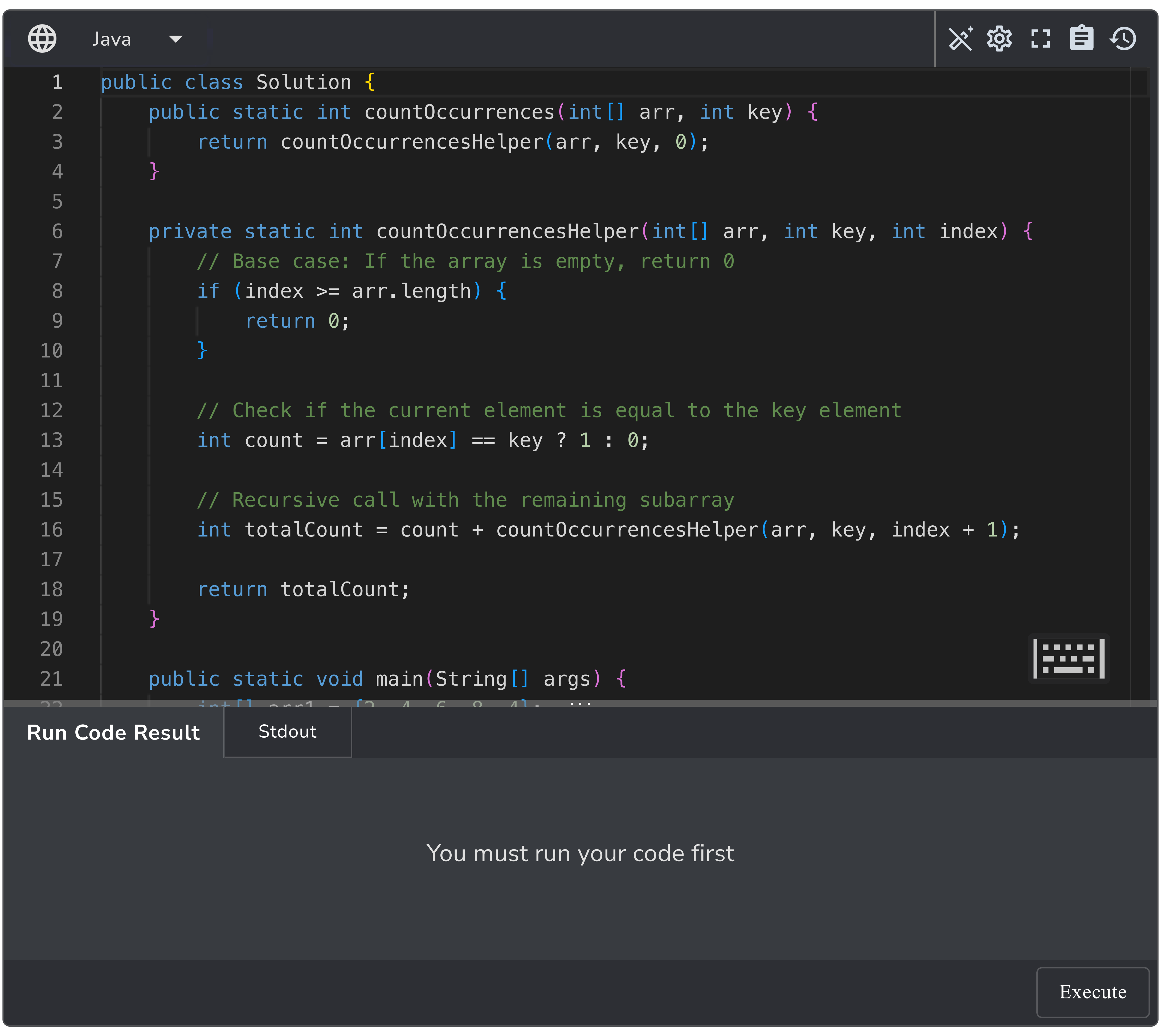
Solution

1. Base Case:
   - If the array is empty, return 0 because there are no occurrences of the key element.
2. Recursive Case:
   - Compare the first element of the array with the key element.
     - If they are equal, increment the count by 1.
   - Make a recursive call with the remaining subarray.
   - Add the count from the recursive call to the current count and return the total count.


2
input: [2,4,6,8,4]
countOccurrences(arr, 4)
countOccurrencesHelper(arr, 4, 0)
2
countOccurrencesHelper(arr, 4, 0)
4 == 2 ? 1: 0
countOccurrencesHelper(arr, 4, 1)
0 + 2
countOccurrencesHelper(arr, 4, 0)
4 == 2 ? 1: 0
countOccurrencesHelper(arr, 4, 2)
1 + 1
countOccurrencesHelper(arr, 4, 2)
4 == 6 ? 1: 0
countOccurrencesHelper(arr, 4, 3)
0 + 1
countOccurrencesHelper(arr, 4, 3)
4 == 8 ? 1: 0
countOccurrencesHelper(arr, 4, 1)
0 + 1
countOccurrencesHelper(arr, 4, 5)
countOccurrencesHelper(arr, 4, 4)
4 == 4 ? 1: 0
1 + 0
countOccurrencesHelper(arr, 4, 5)
5 >= 5
return 0
0

Code

Here is the code for this algorithm:



```java
public class Solution {
    public static int countOccurrences(int[] arr, int key) {
        return countOccurrencesHelper(arr, key, 0);
    }

    private static int countOccurrencesHelper(int[] arr, int key, int index) {
        // Base case: If the array is empty, return 0
        if (index >= arr.length) {
            return 0;
        }

        // Check if the current element is equal to the key element
        int count = arr[index] == key ? 1 : 0;

        // Recursive call with the remaining subarray
        int totalCount = count + countOccurrencesHelper(arr, key, index + 1);

        return totalCount;
    }

    public static void main(String[] args) {
```

You must run your code first

Time and Space Complexity

The time complexity of the algorithm is $O(N)$ because, in the worst case, it needs to examine all elements of the array once. The space complexity of the algorithm is $O(N)$ because the recursive calls consume additional space on the call stack, and in the worst case, the depth of the call stack is proportional to the size of the array.